

在 JavaScript 中，对数组[6, 3, 1, 5, 2, 4]按数值进行从大到小排序，并将结果以分号相隔拼成一个字符串，并使用 alert 弹出该字符串。

```
<script>
var arr = [6, 3, 1, 5, 2, 4];
arr.sort(function(x, y) { //3 分
return y-x;
});
var str = arr.join(';');
//2 分
alert(str);
</script>
```

26、在 JavaScript 中，编写一个用户(User)类，属性有名字 (name)、年龄 (age)，方法有 show()，在方法中使用 alert 弹出用户的名字和年龄。定义一个对象并调用方法。

```
<script>
function User(name, age) { //2 分
this.name = name;
this.age = age;
}
User.prototype.show = function() { //2 分
alert(this.name + ", " + this.age);
}
var user = new User('test', 20); //1 分
user.show();
</script>
```

27、请详细描述 Servlet 的生命周期。

Servlet 有良好的生存期的定义，包括加载和实例化、初始化、处理请求以及服务结束。这个生存期由 javax.servlet.Servlet 接口的 init、service 和 destroy 方法表达。描述 Servlet 请求的流

程。请 HTTP 协议基于请求响应模式，客户端向服务器发送一个请求，服务器则以一个状态行为作为响应。

(1) 第 1 次访问 servlet 时，实例化 Servlet 类并调用 init 方法进行初始化。

(2) 每次访问 servlet 时，调用 service 方法，service 方法根据请求的类型，再调用相应的

doGet 或 doPost()方法。

(3) 当 Servlet 容器关闭或重启时，调用 destroy 方法销毁 servlet 对象。

(4) Servlet 默认是单实例的。

28、假设有一个过滤器，其全限定类名为：com.web.EncodingFilter，要求它能拦截所有的请求，且传入字符集参数 charset。请在 web.xml 文件中注册该过滤器。

```
<filter>
    <filter-name>EncodingFilter</filter-name>
<filter-class>com.web.EncodingFilter</filter-class>
<init-param>
    <param-name>charset</param-name>
    <param-value>UTF-8</param-value>
</init-param>
</filter>
<filter-mapping>
    <filter-name>EncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

如果一个 Servlet 类其全限定名为： com.kzw.web.LoginServlet，希望能处理请求 http://localhost:8080/demo/servlet/LoginSvt。其中 demo 为项目名，请在 web.xml 文件中注册此 Servlet。

```
<servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.kzw.web.LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/servlet/LoginServlet</url-pattern>
</servlet-mapping>
```

五、综合题（本大题共 2 小题，共 35 分）

编写一个字符集过滤器，解决表单 POST 方式提交的中文乱码。要求：

- 1)、可以配置某种字符集，如 UTF-8 或 GBK; (7 分)
- 2)、在 web.xml 中写出过滤器的配置片段。(8 分)

```
package com.kzw.filter;

public class CharSetFilter implements Filter {
    private String encoding;

    public void destroy() {
    }

    public void doFilter(ServletRequest req,
        ServletResponse resp, FilterChain chain)
        throws IOException, ServletException {
        req.setCharacterEncoding(encoding);
        chain.doFilter(req, resp);
    }
}
```

```

    }

    public void init(FilterConfig config)
throws ServletException {
        encoding = config.getInitParameter("encoding");
    }
}

```

Web.xml 中的配置：

```

<filter>
    <filter-name>CharSetFilter</filter-name>
    <filter-class>com.kzw.filter.CharSetFilter</filter-class>
    <init-param>
        <param-name>encoding</param-name>
        <param-value>UTF-8</param-value>
    </init-param>
</filter>
<filter-mapping>
    <filter-name>CharSetFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

```

29、编写一个用户注册的 JSP 页面，效果图如下：

- 1)、使用 html 编写出上图效果的页面。
- 2)、提交表单时使用 JS 验证：①账号为 4-8 个字符；②密码不能为空，且二次密码必须相同。（可使用 jquery 获得元素）

```

<!DOCTYPE html>
<html><head><title>用户注册</title></head><body><center>
<h1>用户注册</h1><hr>
<form action="#" method="post" onsubmit="return checkval()">
    <table><tr>
        <td align="right">账号： </td>
        <td><input type="text" name="name" id="name"></td>
    </tr><tr>
        <td align="right">密码： </td>
        <td><input type="password" name="passwd" id="passwd"></td>
    </tr><tr>
        <td align="right">确认密码： </td>
        <td><input type="password" name="passwd2" id="passwd2"></td>
    </tr>
<tr align="center">
    <td colspan="2">
        <input type="submit" value="确定">
    </td>
</tr>

```

```

        <input type="reset" value="取消">
    </td>
</tr>
</table>
</form></center></body></html>
<script>
function checkval() {
    var nameElm = document.getElementById('name');
    var passwdElm = document.getElementById('passwd');
    var passwd2Elm = document.getElementById('passwd2');
    if(!/^\\w{4,8}$/.test(nameElm.value)) {
        alert('账号必须为[4,8]位字符');
        nameElm.focus();
        return false;
    }
    if(passwdElm.value == '') {
        alert('密码不能为空');
        passwdElm.focus();
        return false;
    }
    if(passwdElm.value != passwd2Elm.value) {
        alert('两次密码不相同');
        passwd2Elm.value = '';
        passwd2Elm.focus();
        return false;
    }
    return true;
}
</script>

```

编写一个增加人员的 JSP 页面，效果图如下：

- 1)、使用 html 编写出上图效果的页面
- 2)、使用 javascript 验证昵称、密码不能为空，且密码长度不能少于 5

```

<!DOCTYPE html>
<html>
    <head>
        <title>增加人员</title>
        <script>
            function checkval() {
                var nameElm = document.getElementById('name');
                var passElm = document.getElementById('pass');
                if(nameElm.value == '') {

```

```
        alert('昵称不能为空');
        nameElm.focus();
        return false;
    }
    if(passElm.value == "") {
        alert('密码不能为空');
        passElm.focus();
        return false
    }
    if(passElm.value.length < 5) {
        alert('密码长度不能少于 5');
        passElm.focus();
    }
    return false
    }
    return true;
}
</script>
</head>
<body>
<center>
    <h1>增加人员</h1><hr>
    <form action="#" method="post"
onsubmit="return checkval()">
        <table>
            <tr>
                <td align="right">昵称: </td>
                <td>
                    <input type="text" name="name" id="name">
                </td>
            </tr>
            <tr>
                <td align="right">密码: </td>
                <td>
                    <input type="text" name="pass" id="pass">
                </td>
            </tr>
            <tr align="center">
                <td colspan="2">
                    <input type="submit" value="提交">
                    <input type="reset" value="取消">
                </td>
            </tr>
        </table>
    </form>
</center>
```

```
</body>
</html>
```

30、编写一个 Servlet，获得用户信息列表，然后跳转到 user_list.jsp 页面，在此页面中使用 JSTL 标签展示用户列表信息。假设 userService 对象有方法 listAll()返回用户列表 List<User>，用户信息：ID(id)、姓名(name)、密码(passwd)。

- 1)、编写 Servlet 类的 doPost(HttpServletRequest req, HttpServletResponse)方法；
- 2)、在 web.xml 文件中注册此 Servlet 类；(类名： com.web.UserListServlet)；
- 3)、使用 JSTL 标签编写展示用户列表 JSP 页面中的 table 部分。

1)、Servlet 类的方法：

```
public void doPost(HttpServletRequest req,
                    HttpServletResponse resp) throws Exception {
    List<User> list = userService.listAll();
    req.setAttribute("list", list);
    req.getRequestDispatcher("/user_list.jsp").forward(req, resp);
}
```

2)、web.xml 中注册 servlet：

```
<servlet>
<servlet-name>UserListServlet</servlet-name>
<servlet-class>com.web.UserListServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>UserListServlet</servlet-name>
<url-pattern>/userList</url-pattern>
</servlet-mapping>
```

3)、在 user_list.jsp 页面中展示：

```
<table>
<tr>
<td>序号</td>
<td>ID</td>
<td>姓名</td>
<td>密码</td>
</tr>
<c:forEach var="s" items="${list}" varStatus="idx">
<tr>
<td>${idx.index+1}</td>
<td>${s.id}</td>
<td>${s.name}</td>
<td>${s.passwd}</td>
</tr>
</c:forEach>
</table>
```